

PH Stocks Advisor — Agent Developer Onboarding

Audience: a developer or architect joining the project who needs to understand how the multi-agent analysis framework is built, how its parts talk to each other, and exactly what happens — class by class, function by function — when a user asks for a stock analysis.

Scope: architecture and execution flow. This is not an API reference (see the README's endpoint and environment tables) and it deliberately avoids line-by-line code narration.

1. What this system is

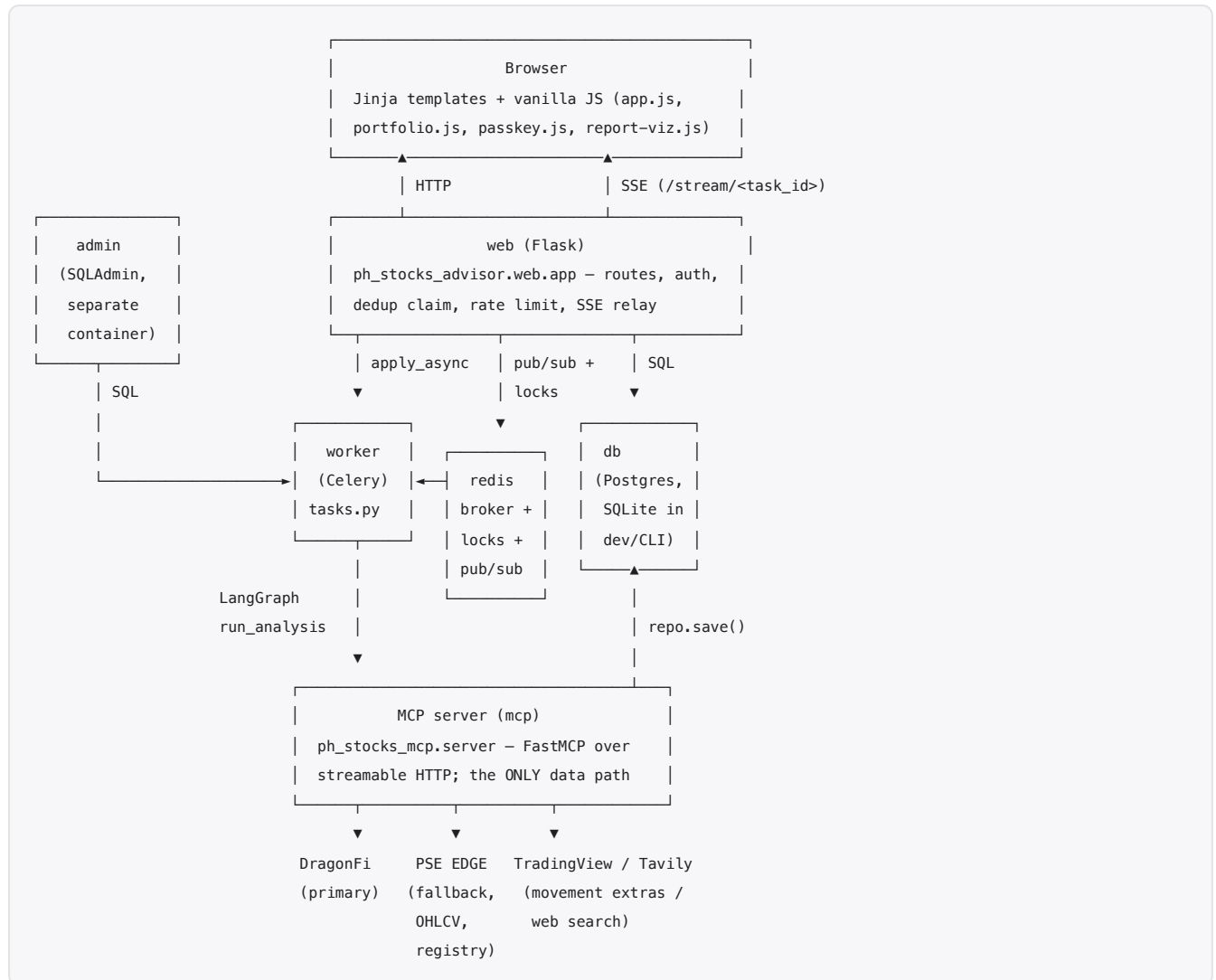
PH Stocks Advisor analyses Philippine Stock Exchange (PSE) tickers with a **LangGraph multi-agent pipeline**: six specialist LLM agents analyse one dimension each (price, dividends, movement, valuation, controversy, sentiment) in parallel, and a **consolidator agent** synthesises them into a single report with a **0–100 verdict score** rendered on an AVOID → WAIT → BUY scale. Reports are cached per trading day, streamed live to the browser, and persisted for history.

Three design commitments shape almost every module:

1. **LLMs judge, code computes.** Every number that matters (payout ratios, weighted verdict score, fair-value inputs) is computed deterministically in Python and handed to the LLM. The LLM writes prose and per-dimension sub-scores against explicit rubrics; it never does arithmetic the app depends on.
 2. **Agents only see what the payload carries.** Prompts may only reference fields that exist on the Pydantic model serialised into them (the *agent data contract*). Facts like REIT status are always data, never LLM inference.
 3. **Degrade honestly.** A missing data source becomes a disclosed “DATA UNAVAILABLE” gap that is excluded from the score — never a fabricated number, a false rejection, or a crashed task.
-

2. General architecture

2.1 Runtime topology (Docker Compose services)



The seventh compose service, `advisor`, is a one-shot CLI container that runs `ph-advisor <SYMBOL>` against the same stack — useful for smoke-testing the pipeline without the web UI.

2.2 Package layout and responsibilities

Package	Responsibility	Key modules
ph_stocks_advisor/graph/	LangGraph orchestration	workflow.py — state schema, node factories, run_analysis()
ph_stocks_advisor/agents/	LLM agents + prompts	specialists.py (6 agents), consolidator.py, portfolio.py, prompts.py
ph_stocks_advisor/data/	Data acquisition	tools.py (façade), mcp_client.py, models.py (Pydantic), services/*, clients/*
ph_stocks_mcp/	MCP server	server.py — FastMCP tool definitions wrapping the services
ph_stocks_advisor/infra/	Cross-cutting infra	config.py (Settings + LLM factory), repository*.py (persistence), trading_calendar.py, tracing.py (Langfuse)
ph_stocks_advisor/web/	Web app + async plumbing	app.py (routes), tasks.py (Celery), progress.py (SSE), rate_limit.py, auth.py, passkey.py, templates + static JS
ph_stocks_advisor/export/	Report rendering/export	formatter.py (parse_sections), html.py, pdf.py
admin/	Back-office	Separate SQLAdmin app (users, passkeys, reports)

2.3 How the components communicate

- **Browser ↔ web:** ordinary HTTP for pages and JSON APIs; **Server-Sent Events** on /stream/<task_id> for live progress (with /status/<task_id> polling as fallback).
- **web ↔ worker:** never direct. The web process enqueues a Celery task (analyse_stock.apply_async(..., task_id=...)) on the Redis broker; results and progress come back through Redis, not through return values.
- **worker ↔ web (progress):** the worker's graph nodes call web.progress.publish_progress(task_id, step, ...) , which writes a JSON event to a Redis **pub/sub channel** and stores a **state snapshot** key. The web SSE endpoint (subscribe_progress) replays the snapshot to late joiners, then streams live events — this is what lets a second browser “join” an in-flight analysis mid-run.
- **Everything ↔ market data:** all data fetching goes through the **MCP server**. The façade data/tools.py exposes validate_symbol, fetch_stock_price, fetch_dividend_info, fetch_price_movement, fetch_fair_value, fetch_controversy_info, fetch_sentiment_info; each dispatches through data/mcp_client.py (a synchronous wrapper that owns a background asyncio session, rebuilds it on transport failure, and translates the server's SymbolNotFoundError: marker back into a typed exception). There is **no in-process fallback** — an unset MCP_SERVER_URL is a hard configuration error.
- **web/worker ↔ database:** through infra.repository.AbstractReportRepository, with repository_sqlite.py and repository_postgres.py implementations selected by Settings.db_backend. Both run **idempotent schema migrations** at initialize() .
- **Coordination state (Redis keys):** analysis:inflight:<SYMBOL> (same-ticker dedup lock, SET NX), analysis:task:<task_id> (reverse mapping for cancel), ratelimit:<user>:<day> counters (atomic Lua reserve/release in web/rate_limit.py), analysis:progress:* (SSE snapshots), portfolio:inflight:<user>:<symbol> .

2.4 The LLM layer (provider-agnostic, per-agent)

infra/config.py owns model selection:

- build_chat_model(spec) turns a [provider:]tier string (provider ∈ openai | anthropic, tier ∈ large | medium | small) into a LangChain BaseChatModel — ChatOpenAI (with temperature) or ChatAnthropic (with max_tokens, deliberately **no** temperature: current Claude models reject one). Six env vars ({OPENAI, ANTHROPIC}_MODEL_{LARGE, MEDIUM, SMALL}) define what each tier means.
- AGENT_LLM_SPECS maps every agent (price_agent ... sentiment_agent, consolidator, portfolio) to a spec, overridable per agent via LLM_<AGENT> env vars. Different agents can use different providers **in the same run** (e.g. consolidator on anthropic:large, specialists on openai:small).
- get_agent_llm(name) resolves an agent's model; get_llm() / get_mini_llm() survive as back-compat shims.

Because everything downstream depends only on BaseChatModel, agents never know or care which provider is active (Liskov substitution in practice).

3. Execution walkthrough: one analysis, end to end

The scenario: an authenticated user types **AREIT** and clicks *Analyse*.

Step 0 — Gatekeeping in the web tier

`POST /analyse` lands in the `analyse` route of `ph_stocks_advisor.web.app`:

1. **Identity:** `auth.get_current_user()` supplies `{oid, email, user_type}` (populated at login by the passkey blueprint `web/passkey.py` or the OAuth blueprint `web/auth.py`).
2. **Trading-session gate:** helps delegating to `infra.trading_calendar` decide freshness. A report is *fresh* if created after `last_trading_close()` (the most recent trading-day 3:00 PM PHT close; weekends roll back to Friday). During market hours (`is_market_open()`), no new run starts — the newest report is served with a “refreshes after close” note.
3. **Cache check:** `repository.get_latest_by_symbol(symbol)` — a fresh report short-circuits to `{"status": "cached", "report_id": ...}`.
4. **Rate limit:** `rate_limit.reserve(redis, user_id, limit)` atomically reserves a slot in the per-user daily counter (Lua script; elevated users bypass). Denial releases the claim.
5. **Same-ticker dedup:** the route pre-generates a task id, then attempts `redis.set("analysis:inflight:AREIT", task_id, nx=True, ex=600)`. Losing the claim means another user's run is in flight → respond `{"status": "joined", "task_id": <winner's>}` so this browser streams the winner's progress. Winning the claim proceeds to `analyse_stock.apply_async(args=[symbol], kwargs={"user_id": ...}, task_id=task_id)` — dispatching under the *same* pre-generated id is what makes joining race-free.

Step 1 — The browser attaches to the stream

`static/app.js` receives `started / joined` and opens `EventSource("/stream/<task_id>")`. The web SSE generator `progress.subscribe_progress(task_id)` first emits the stored snapshot (so late joiners see the current step immediately), then relays pub/sub events. A polling fallback hits `/status/<task_id>`, which reads the Celery `AsyncResult` and, on success, returns `verdict`, `score`, and `report_id`.

Step 2 — The worker starts the graph

`web/tasks.py :: analyse_stock` (Celery task, `bind=True`) wraps everything in `try/except/finally`:

- calls `graph.workflow.run_analysis(symbol, task_id=..., user_id=...)`;
- on success saves and publishes (step 5 below);
- `finally:` calls `_clear_inflight_lock(symbol, task_id)` so the dedup lock never leaks.

`run_analysis` builds the graph via `_build_graph_impl`:

- resolves the consolidator LLM (`get_llm()`) and, per specialist node, its own model via `get_agent_llm(node_name)` — unless a test injected `mini_llm`, which overrides all six;
- assembles a `StateGraph(GraphState)` with the topology `validate → [6 specialists in parallel] → consolidator`;
- attaches Langfuse tracing config (`infra.tracing.build_langfuse_config`) when keys are set.

`GraphState` (a `TypedDict`) is worth understanding — it encodes the concurrency rules:

- `error: Annotated[str | None, _keep_first_error]` — a **reducer** channel, because six parallel nodes may fail in the same superstep; the first error wins deterministically.
- `data_gaps: Annotated[list[str], operator.add]` — parallel-safe accumulation of dimensions that produced no real data.
- one typed slot per analysis (`price_analysis: PriceAnalysis | None, ...`) so fan-out nodes write disjoint keys.

Step 3 — Validation

`_make_validate_node` publishes `STEP_VALIDATING`, then calls the façade's `validate_symbol`. Behind MCP, `dragonfi.validate_pse_symbol` runs a three-source check:

1. DragonFi's full listing universe (`_fetch_all_stock_codes`, cached only when non-empty — an empty result is never pinned);

2. the DragonFi profile probe (covers preferred shares and fresh IPOs);
3. **PSE EDGE's own registry** (`pse_edge.symbol_exists` , tri-state) as independent fallback.

The error semantics are strict: a definitive `SymbolNotFoundError` requires the DragonFi universe to be loaded; if no source can answer, `SymbolValidationUnavailableError` produces a “temporarily unavailable, try again” — an upstream outage is never reported as “not listed”. The validate node catches `SymbolNotFoundError` into the state's `error` channel and any other exception into a clean retryable error; nothing crashes the Celery task.

Step 4 — Parallel specialists

For each `AGENT_REGISTRY` entry (`node_name` , `state_key` , `agent_class`) , `_make_specialist_node` produces a node that:

1. short-circuits if `state["error"]` is set (validation failed);
2. runs `agent_class(llm).run(symbol)` — e.g. `DividendAgent.run` : - fetches `DividendInfo` via `fetch_dividend_info` (MCP → `services/dividend.py`), which computes the NI-based `payout_ratio` **and** the cash-based `fcf_payout_ratio` (dividends ÷ latest FCF) deterministically, plus PSE EDGE declared-dividend announcements; - raises `EmptyAgentDataError` if the payload is genuinely empty (`_is_empty_dividend_info`); - otherwise formats `DIVIDEND_ANALYSIS_PROMPT` with `data.model_dump_json()` and invokes the LLM for a prose analysis;
3. on `EmptyAgentDataError` or any exception, **does not abort**: it substitutes `_fallback_analysis(state_key, symbol, transient=...)` — a placeholder whose text starts with `DATA UNAVAILABLE` — and appends the dimension to `data_gaps` ;
4. publishes `STEP_AGENTS` with the agent name so the UI ticks the per-agent progress.

The other five specialists follow the same shape with their own service payloads: `StockPrice` (with EDGE stockData snapshot fallback), `PriceMovement` (EDGE OHLCV + TradingView + candlestick heuristics), `FairValueEstimate` (Graham-number inputs, EDGE audited-financials fallback, explicit `is_reit`), `ControversyInfo` (spike detection + news), `SentimentInfo` (Tavily global events + BSP policy-rate search + explicit `is_reit`).

Step 5 — Consolidation and scoring

`_make_consolidate_node` :

1. skips entirely on upstream `error` ;
2. aborts with a clean error if **every** dimension is in `data_gaps` (systemic-failure guard — nothing real to consolidate);
3. builds the Pydantic `AdvisorState` (the six analyses + sorted `data_gaps`) and calls `ConsolidatorAgent.run` .

`ConsolidatorAgent` does the heart of the product:

- formats `CONSOLIDATION_PROMPT` (structure rules, REIT context, MISSING-DATA rules, precision-matches-confidence rules, per-dimension scoring rubric);
- invokes `llm.with_structured_output(ConsolidationResponse)` — the typed response carries the summary, a binary verdict claim, and six optional 0–100 sub-scores;
- **force-nulls** sub-scores for gap dimensions (`_GAP_SCORE_FIELDS`) so the LLM cannot score missing data;
- computes `_weighted_score` — a weighted average using env-tunable `SCORE_WEIGHT*` settings, renormalised over present dimensions;
- **derives** the binary verdict from the score (`score >= buy_score_threshold` → `BUY`), overriding the LLM's claim so the meter, band label, and chips can never contradict;
- falls back to regex verdict extraction (with a fixed 75/25 score) for models without structured output.

The returned `FinalReport` carries symbol, verdict, `score` , the consolidated summary, and the six section texts.

Step 6 — Persistence and completion

Back in `tasks.analyse_stock` : `ReportRecord.from_final_report(report)` → `repo.save()` , `repo.add_user_symbol(user_id, symbol)` (drives the per-user sidebar), then the terminal event `publish_progress(task_id, STEP_SAVING, done=True, verdict=..., score=..., report_id=...)` . The `finally` block clears the inflight lock. The SSE terminal event reaches every attached browser; `app.js` navigates to `/report/<symbol>` .

Step 7 — Rendering

The `report` route in `web/app.py` :

- maps `record.score` through `models.score_band` (80/60/40/20 bounds → STRONG BUY / BUY / WAIT / DON'T BUY / AVOID) and positions the meter marker at `score%` ;
- `export/formatter.parse_sections(summary)` splits the summary into `(title, body)` cards and enforces display invariants at this single choke point: standalone verdict lines are stripped, verdict-labelled sections are retitled “Why This Verdict”, document-title headings and empty-bodied sections are dropped (these render-time rules also repair already-cached reports);
- the Jinja template renders the verdict panel (word + band-coloured score + red→green meter), the metric table, and agent cards; `report-viz.js` decorates inline ↑/↓ percentages; chips elsewhere (sidebar/marquee/history) come from the shared `verdict_chip` Jinja global and its JS mirror `bandChip` .

Secondary flow: portfolio advisory (elevated users)

`POST /api/portfolio-analyse/<symbol>` (in `web/app.py`) requires an elevated user and a saved `HoldingRecord` , enforces its own per-day cooldown, and either chains a base analysis (`analyse_stock.apply_async(link=[portfolio_task])`) with a pre-assigned callback task id) or dispatches `portfolio_analyse_stock` directly. `agents/portfolio.py :: PortfolioAgent` combines the user's cost basis and P/L with the base report to produce a private hold/accumulate/trim advisory, stored via `repo.save_portfolio_report` .

4. Resilience model (what happens when things fail)

Failure	Behaviour	Where
DragonFi fully down (real incident: HTTP 515 on every endpoint)	Validation, price, valuation, financial trends, and REIT status all fall back to PSE EDGE (registry search, stockData scrape, financial-reports scrape, name/description REIT check). Dividend yield/payout become disclosed gaps.	<code>clients/dragonfi.py</code> , <code>clients/pse_edge.py</code> , <code>services/*</code>
One data dimension empty (e.g. a stock that pays no dividends)	<code>EmptyAgentDataError</code> → fallback analysis + <code>data_gaps</code> ; the score renormalises; the report states the gap.	<code>graph/workflow.py</code> , <code>agents/consolidator.py</code> , <code>prompts</code>
Every dimension fails (LLM key dead, total outage)	Consolidate node aborts with a clean error; nothing junk is cached.	<code>_make_consolidate_node</code>
Ambiguous listing status	Tri-state semantics: transient <code>SymbolValidationUnavailableError</code> vs definitive <code>SymbolNotFoundError</code> ; EDGE-no-match alone is never definitive (its autocomplete omits preferred shares).	<code>validate_pse_symbol</code>
Duplicate concurrent requests	<code>SET NX</code> claim; losers join the winner's SSE stream; failure paths release the claim and the rate-limit slot.	<code>web/app.py</code>
Worker crash mid-run	Inflight lock has a 10-minute TTL; <code>finally</code> clears it eagerly.	<code>web/tasks.py</code>

5. Extending the system

- **New specialist agent:** add a Pydantic payload to `data/models.py` , a service + MCP tool, an agent class in `agents/specialists.py` , a prompt, and register one tuple in `AGENT_REGISTRY` — the graph, per-agent LLM specs (add a key to `AGENT_LLM_SPECS`), fallback builders (`_fallback_analysis`), and UI cards pick it up from there.
- **New export format:** subclass `export.formatter.OutputFormatter` and register it.
- **New data source / fallback:** add a client under `data/clients/` and wire it in the relevant service; keep tri-state semantics for anything that gates user-visible rejection.
- **Prompt changes:** respect the agent data contract (only reference payload fields; state the negative case explicitly) and never let the model emit a verdict label into prose — the score pipeline owns the verdict.

6. Development workflow expectations

- **Tests:** `pytest -m "not integration"` must stay green; template/JS contracts (e.g. `#portfolio-btn[data-symbol]`, `.section-body`, chip classes) are pinned by tests — redesigns must update them consciously.
- **Gates:** `ruff check` + `ruff format` + `pyright` (basic mode) match CI exactly.
- **Verification culture:** UI changes are verified against the **Docker image** (the running container serves site-packages, not the repo — rebuild to see changes) with Playwright screenshots; pipeline changes get a real one-ticker run.
- **Branching:** work lands on `develop`; merge-commit-only promotion to `main`; the main pipeline path-gates image builds and pushes to GHCR.
- **Versioning:** fully automatic and tag-driven. Each merge to `main` that rebuilds the app image runs the `bump` job — `next = max(latest vX.Y.Z tag patch+1, pyproject.toml floor)` — pushes an annotated tag only (no commit), bakes that version into `pyproject.toml` before the build, tags the image `latest + X.Y.Z`, then verifies the published image reports the same version. Raise the `pyproject.toml` floor for a minor/major. `GET /version` reports the running build.
- **Dependency locking:** `uv pip compile --extra=postgres --upgrade -o requirements.txt pyproject.toml` — the `postgres` extra is mandatory or `psycopg2` silently disappears.